Solution: Word Search

Problem Statement

Given an m x n grid of characters board and a string word, return true if the word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

Input: word="ABCCED", board:

```
    { 'A', 'B', 'C', 'E' },
    { 'S', 'F', 'C', 'S' },
    { 'A', 'D', 'E', 'E' }
```

Output: true Explanation: The word exists in the board:

-> { **'A'**, **'B'**, **'C'**, 'E' },

-> { 'S', 'F', **'C'**, 'S' },

-> { 'A', **'D'**, **'E'**, 'E' }

Example 2:

Input: word="SEE", board:

```
    { 'A', 'B', 'C', 'E' },
    { 'S', 'F', 'C', 'S' },
    { 'A', 'D', 'E', 'E' }
```

Output: true Explanation: The word exists in the board:

-> { 'A', 'B', 'C', 'E' },

-> { 'S', 'F', 'C', **'S'** },

-> { 'A', 'D', **'E'**, **'E'** }

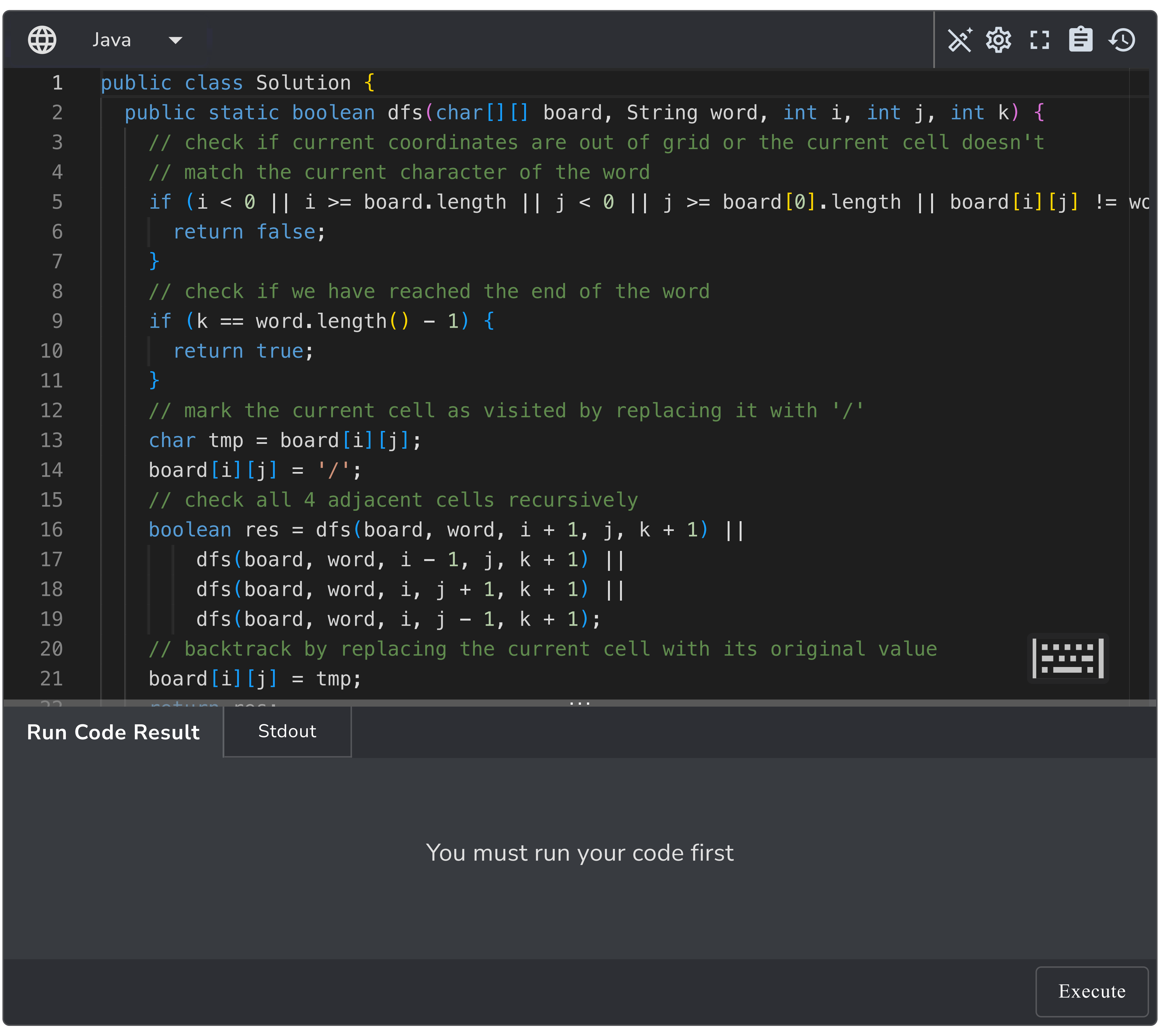
Solution

The basic approach to solving the word search problem using backtracking is to start at the first character of the word and check all 8 adjacent cells in the grid to see if any of them match the next character of the word. If a match is found, mark the cell as visited and recursively check the next character of the word in the adjacent cells of the newly visited cell. If the entire word is found, return true. If no match is found, backtrack to the previous cell and try a different path. Repeat this process until the entire grid has been searched or the word is found.

Code

This function takes a 2D list `board` and a string `word` as input, and returns `True` if the word can be found in board and `False` otherwise. It uses a helper function `dfs` which takes 4 additional parameters: `i` and `j` are the current coordinates of the cell that is being visited, `k` is the index of the current character of the word being matched, and `board` and `word` are the inputs passed to the main function.

The dfs function uses a helper variable `tmp` to store the current value of the cell before it is marked as visited. This is done so that we can backtrack later. It then uses recursion to check if the next character of the word exists in the 4 adjacent cells, and it will mark the cell as visited and move to next index of the word by incrementing `k` by 1. If the next character is found, the function returns `true`, if not it backtracks to the previous cell, and continues the search in different path. If the entire word is found, the function returns `True`, otherwise it returns `False` after searching the entire grid.



```java
public class Solution {
  public static boolean dfs(char[][] board, String word, int i, int j, int k) {
    // check if current coordinates are out of grid or the current cell doesn't
    // match the current character of the word
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length || board[i][j] != wo
      return false;
    }
    // check if we have reached the end of the word
    if (k == word.length() - 1) {
      return true;
    }
    // mark the current cell as visited by replacing it with '/'
    char tmp = board[i][j];
    board[i][j] = '/';
    // check all 4 adjacent cells recursively
    boolean res = dfs(board, word, i + 1, j, k + 1) ||
        dfs(board, word, i - 1, j, k + 1) ||
        dfs(board, word, i, j + 1, k + 1) ||
        dfs(board, word, i, j - 1, k + 1);
    // backtrack by replacing the current cell with its original value
    board[i][j] = tmp;
```

You must run your code first

Time Complexity

The time complexity of the 'exist' function is $O(4^n)$, where n is the length of the word. This is because the function uses a depth-first search (DFS) algorithm to traverse the board, and for each cell in the board, it checks all 4 adjacent cells recursively. The worst-case scenario is when the word is found in the last cell of the board, and in that case, the function will have to check all possible paths from the starting cell to the last cell, which is 4^n possible paths.

Space Complexity

The space complexity of the `exist` function is $O(n)$, where n is the length of the word. This is because the function uses a DFS algorithm, and the maximum depth of the recursion tree is n. In other words, the maximum number of function calls that will be stored on the call stack at any point in time is n.

← Previous
Word Search (Medium)
Next →
Sudoku Solver (Hard)
☐ Mark as Completed